

Módulo 06 — Orquestração com LangGraph.js

Cada slide da apresentação explicado em linguagem simples: termos, o que falar e o que responder.

1. Slide a slide

Slide a slide

Este guia explica cada slide da apresentação em linguagem simples: o que significa cada termo, o que falar e o que responder se perguntarem. Leia uma vez inteiro antes da aula; durante a aula, as notas do PPTX bastam.

Se só der tempo de entender uma coisa: um **grafo** é um fluxograma que o computador executa: caixinhas (os passos) ligadas por setas (a ordem), com setas que escolhem o caminho conforme a situação. O **checkpoint** é o "salvar jogo": a cada passo, o estado é gravado, e se o programa cair, ele continua do último ponto salvo em vez de começar do zero.

Antes da aula (uns 20 minutos, quase tudo esperando o modelo):

1. `pnpm -F @mentoria/curador m06:grafo:solucao -- --nova --falhar` (vários minutos): roda o curador inteiro e "cai" de propósito no último passo.
2. `pnpm -F @mentoria/curador m06:grafo:solucao` (segundos): retoma do checkpoint. Guarde os dois relatórios (o segundo sobrescreve `saidas/m06-relatorio.md`; copie o primeiro antes).
3. Abra `projeto-final/saidas/m06-grafo.mmd`, copie o conteúdo e cole em <https://mermaid.live> para ver o desenho do grafo. Deixe a aba aberta.
4. `pnpm -F @mentoria/ex06-langgraph demo:grafo` e `demo:checkpoint` (instantâneos, não usam modelo): rode uma vez para ver a saída.

Slide 1 — Capa

Diga: *"Na aula passada, o time de agentes era uma função que chamava os papéis em ordem. Hoje ele vira um grafo: dá para desenhar, escolher o caminho pelo estado e retomar se cair."*

Slide 2 — Hoje

Os objetivos. Não leia a lista.

Slide 3 — A frase da aula

Um workflow vira grafo quando você precisa retomar, desviar ou pausar.

- **Workflow:** um fluxo de passos definido pelo código (Aula 4).
- **Retomar:** continuar de onde parou depois de uma falha.
- **Desviar:** seguir caminhos diferentes conforme a situação (por exemplo, confiança baixa vai para um humano).
- **Pausar:** esperar uma pessoa aprovar (é a Aula 7).

A segunda linha é tão importante quanto a primeira: se o fluxo é curto e sempre igual, uma função comum resolve. LangGraph é uma ferramenta para um problema específico, não um passo obrigatório.

Slide 4 — O que faltava no time da Aula 5

Cada linha é um problema real que apareceu na execução da Aula 5:

- **Caiu no último passo:** o time levou ~14 minutos. Se a publicação falhasse no fim, era preciso refazer tudo, incluindo as 32 classificações.
 - **Confiança baixa:** no pipeline, todo item seguia o mesmo caminho. No grafo, um item incerto **desvia** para uma fila de revisão humana.
 - **Edição sem regulação nem mercado:** na Aula 5, a regra "os 10 de maior confiança" deixou duas categorias de fora (quase todas as confianças ficaram entre 0,95 e 0,99, e o empate foi decidido pela ordem das fontes). No grafo, o passo de seleção tem uma **cota**: no máximo 3 por categoria.
 - **Entender o fluxo:** no grafo, o próprio LangGraph desenha o fluxo.
-

Slide 5 — Divisória "1 · As peças de um grafo"

Passe direto.

Slide 6 — Estado, nós e arestas

Os três conceitos centrais. Use a analogia do **fluxograma de processo**, só que executável:

- **Estado:** a "pasta" com os papéis do caso, que passa de mesa em mesa. Tem campos com tipos definidos (com Zod, como na Aula 2): a lista de candidatos, os itens avaliados, a edição...
- **Nó:** uma mesa, ou seja, uma função. Recebe a pasta e devolve **só o que mudou** ("acrescentei a categoria"), não a pasta inteira.
- **Aresta:** a seta. "Depois desta mesa, vá para aquela."
- **Aresta condicional:** o losango do fluxograma. Uma função olha a pasta e decide: "cobrança vai para o financeiro, problema técnico vai para o suporte, confiança baixa vai para uma pessoa".

O código mostra o grafo do **exercício** (triagem de chamados). `START` é o começo; `addNode` cria um passo; `addEdge` liga dois passos; `addConditionalEdges` liga um passo a vários possíveis, com a função que escolhe.

Ponto importante: a função que escolhe o caminho (`rotaDeTriagem`) é uma função comum, sem LLM. Dá para testar sozinha. Quem escolhe o caminho é o **código**, a partir do que o LLM classificou.

Slide 7 — Reducers: como juntar atualizações

Reducer é a regra para juntar o valor novo com o antigo.

- **Sem reducer:** o novo **substitui** o antigo. Bom para campos com um valor só (o nome do arquivo publicado).
- **Com reducer:** o LangGraph **combina** os dois. No projeto, o reducer de listas **acrescenta** os itens novos aos antigos.

Por que importa: no curador, as 32 notícias são classificadas **em paralelo** (ao mesmo tempo), e cada classificação escreve no mesmo campo (`avaliados`). Sem reducer, cada uma apagaria a anterior e sobraria só uma.

Analogia: várias pessoas preenchendo a mesma planilha. Com reducer, cada uma acrescenta a sua linha. Sem reducer, cada uma apaga a planilha e escreve só a dela.

Fan-out com Send : *fan-out* é "abrir em leque": uma tarefa vira várias, uma por item.

`Send("classificar", { id })` agenda uma execução do passo `classificar` para cada notícia.

Pergunta provável: *"Em paralelo fica mais rápido?"* Depende de quem atende. Com o modelo local (Ollama), as chamadas entram numa fila, então o ganho de tempo é pequeno. Com um provedor na nuvem, que atende várias chamadas ao mesmo tempo, o ganho é grande.

Slide 8 — Duas armadilhas que pegamos

Duas coisas que aconteceram de verdade ao preparar este módulo:

1. **Aresta condicional logo depois de um fan-out roda uma vez para cada ramo.** Se cada uma das 32 classificações tivesse a sua própria seta "redigir ou não", a decisão rodaria 32 vezes e os itens apareceriam repetidos. **E não dá erro:** só duplica. A solução é um **nó de junção** (`selecionar`): um passo que espera todos os ramos terminarem e roda uma vez só, com tudo pronto. O teste do projeto confere que `selecionar` aparece uma única vez na trajetória.
 2. **O nome de um passo não pode ser igual ao de um campo do estado.** O LangGraph recusa com a mensagem do slide ("já está sendo usado como atributo do estado"). Solução: nomes diferentes.
-

Slide 9 — Divisória "2 · O curador como grafo"

Rode `pnpm -F @mentoria/ex06-langgraph demo:grafo` (instantâneo). Mostre as partes 1 e 2:

1. **O desenho** em texto (formato *mermaid*, uma linguagem para descrever diagramas). As setas tracejadas (`-.->`) são as condicionais.
2. **Quatro chamados**, cada um indo para um lugar. O último ("Acho que fui cobrado errado, talvez") tem confiança 0,4 e vai para um humano.

Depois, troque para a aba do mermaid.live com o grafo do curador.

Slide 10 — O grafo do curador

Percorra o desenho da esquerda para a direita:

- **coletar:** o agente coletor da Aula 5 (lê as fontes pelo MCP). Se não achar nada, o grafo termina.
- **classificar:** um por notícia, em paralelo.
- **selecionar:** o nó de junção. Aplica a regra editorial: confiança de pelo menos 0,9, até 10 itens, no máximo 3 por categoria.
- **redigir:** um por item escolhido. Dentro dele, o ciclo redator $\rightleftharpoons$ revisor da Aula 5.
- **fila_humana:** itens com confiança baixa ou que o guardrail marcou para revisar. **Por enquanto só registra;** na Aula 7, o grafo vai **parar** e esperar uma pessoa decidir.
- **publicar:** chama o servidor de edição pelo MCP.

Ponto importante: os papéis da Aula 5 (coletor, classificador, redator, revisor) entram no grafo **sem mudar uma linha**. O framework só **liga** as funções. Isso facilita trocar de framework no futuro.

Slide 11 — Resultado do grafo

Os números da execução de preparação (os seus podem variar um pouco):

- **coletar**: o agente coletor entregou 32 candidatos.
- **classificar**: 32 chamadas, uma por notícia. As 32 classificações bateram com o gabarito.
- **selecionar**: não chama o modelo, é só código. Escolheu 10 itens, mandou 4 para a fila humana e deixou 15 de fora (pela cota de 3 por categoria ou pelo limite de 10).
- **redigir** $\Rightarrow$ **revisor**: 16 redações e 16 revisões para 10 itens (alguns precisaram de uma segunda tentativa). 9 aprovados; a **n10** foi reprovada duas vezes, como na Aula 5 (o resumo dizia "reduz" e o original diz "mede menos").
- Tempo total: 663 segundos (~11 minutos).

O ponto alto (diga): a cota **resolveu** um problema: regulação, que tinha ficado de fora na Aula 5, agora tem 3 itens. Mas **mercado continua fora**, e o motivo não é empate: o modelo deu **0,95 para todo item de mercado** e **0,98 para as outras categorias**. Como a seleção ordena por confiança, mercado sempre perde a vaga. É um viés do modelo que só apareceu porque olhamos os dados.

Pergunte à turma: como garantir diversidade? Respostas boas: alternar as categorias na escolha (uma de cada, depois a segunda de cada...); comparar a confiança só dentro da mesma categoria; ou ter uma vaga mínima por categoria.

A fila humana pegou **n38** (a notícia com injeção de prompt da Aula 3), **n40** (link para um endereço interno), **n13** e **n17** (as difíceis da Aula 4). Ou seja: os guardrails mandaram para uma pessoa exatamente os casos que devem ir.

Slide 12 — Divisória "3 · Checkpoint e retomada"

Rode `pnpm -F @mentoria/ex06-langgraph demo:checkpoint` (2 segundos). O que aparece:

1. Um processo (veja o número `pid`, que identifica o programa rodando) começa um grafo de três passos: `buscar` $\rightarrow$ `processar` $\rightarrow$ `entregar`.
 2. Ele **morre** no meio de `processar` (simula uma queda: falta de energia, erro, servidor reiniciando).
 3. O checkpoint ficou num **arquivo JSON**.
 4. **Outro** processo (outro `pid`) abre o mesmo arquivo, vê que o próximo passo é `processar` e continua dali. `buscar` não roda de novo.
-

Slide 13 — "Salvar jogo" a cada passo

Checkpoint: o retrato do estado gravado depois de cada passo. **Checkpointeer**: quem grava (na memória, num arquivo, num banco de dados).

Thread (linha de execução): uma execução com memória própria, identificada por um nome (`thread_id`). Pode ser "a edição desta semana", "a conversa com o cliente Ana"...

- **Retomar:** chamar de novo a mesma thread sem entrada nova (`invoke(null, ...)`). O grafo continua do último checkpoint.
- **Memória:** chamar a mesma thread com uma entrada nova. O estado anterior continua lá (no exercício, o histórico de chamados do cliente).
- **Inspecionar:** `getState` mostra o estado atual e o próximo passo; `getStateHistory` mostra todos os checkpoints, do mais novo para o mais antigo.

Onde guardar:

- `MemorySaver` (na memória): some quando o programa termina. Bom para testes.
- **Arquivo JSON:** o que o projeto usa, só para mostrar que um checkpoint é **dado comum**, que dá para abrir e ler.
- **Banco de dados** (SQLite, Postgres): o caminho em produção. Há pacotes oficiais.

Analogia: o "salvar jogo" do videogame. Cada fase concluída grava o progresso; se a energia cair, você volta da última fase salva.

Slide 14 — A retomada no curador

Mostre os dois relatórios lado a lado:

- **1ª execução** (com `--falhar`, que finge que o servidor de edição caiu): 1 coleta, 32 classificações, 16 redações, 16 revisões, 663 segundos. Parou no último passo, `publicar`.
- **2ª execução** (outro processo, mesma thread): **zero** chamadas ao modelo, **1 segundo**. O checkpoint dizia "o próximo passo é `publicar`", e só ele rodou.

Diga: "Sem checkpoint, a falha no último passo custaria mais 11 minutos. Com checkpoint, custou 1 segundo."

O cuidado do rodapé (explique): retomar **roda de novo o passo que falhou**, do começo. Se esse passo fez algo "lá fora" antes de falhar (mandou um e-mail, cobrou um cartão), isso pode acontecer **duas vezes**. Por isso esses passos precisam ser **idempotentes**: rodar duas vezes dá o mesmo resultado que rodar uma. Exemplo: "grave o arquivo `edicao.md`" é idempotente (sobrescreve); "mande um e-mail" não é.

Slide 15 — Viagem no tempo

Viagem no tempo (*time travel*): voltar a um checkpoint antigo, mudar um valor e seguir dali.

O exemplo é o do exercício: o classificador mandou "Não recebi o reembolso" para as perguntas frequentes, mas era cobrança.

1. Procure no histórico o checkpoint logo depois da classificação.
2. Grave a categoria certa nesse ponto (`updateState`). Isso cria um **ramo** novo, como um "salvar como" a partir do save antigo.
3. Continue dali (`invoke(null, ...)`). Só o atendimento roda de novo; o classificador não é chamado.

O passado fica: o caminho errado continua no histórico. Serve para auditoria (o que aconteceu de verdade?) e para criar exemplos de avaliação (Aula 3): cada correção vira um caso de teste.

Mostre a parte 5 da `demo:grafo`.

Slide 16 — Divisória "4 · Alternativas"

Passe direto.

Slide 17 — Não é só LangGraph

Não é um ranking. Cada opção resolve um problema diferente:

- **Código puro:** o que fizemos na Aula 5. Funções, laços e `if`. Sem dependência, fácil de testar. Bom para fluxos curtos.
- **Vercel AI SDK:** a biblioteca que usamos para chamar o modelo desde a Aula 1. Tem o loop de **um** agente (Aula 4). Bom quando o produto é o agente.
- **LangGraph.js:** o assunto de hoje. Bom para fluxos longos, com ramos, falhas e humanos no meio.
- **Mastra:** um framework em TypeScript com *workflows* escritos de forma encadeada (`.then` = "depois", `.branch` = "escolha um caminho", `.parallel` = "ao mesmo tempo"). Também suspende e retoma. Bom para quem quer um pacote completo (agentes, memória, avaliações).
- **OpenAI Agents SDK:** agentes que passam a conversa uns para os outros (*handoff*, Aula 5), com *guardrails* (Aula 3) e aprovação humana. Funciona também com modelos de outros fornecedores. Bom para atendimento com especialistas.
- **Claude Agent SDK:** o "motor" do Claude Code transformado em biblioteca. Vem com ferramentas prontas para ler e editar arquivos e rodar comandos, além de controles de permissão. Bom para automações parecidas com um assistente de programação.

Pergunta provável: "Qual é o melhor?" Depende do problema (próximo slide). E todos mudam rápido: confira a versão e a documentação antes de adotar.

Slide 18 — Como escolher

1. **Precisa retomar, pausar ou voltar no tempo?** Se sim, algo com checkpoint (LangGraph, Mastra). Se não, código puro ou AI SDK.
 2. **O fluxo é fixo ou o modelo decide os passos?** Fixo: workflow. O modelo decide: agente (o framework da Aula 4).
 3. **Quanta abstração a equipe aguenta?** Cada framework tem os próprios conceitos para aprender.
 4. **Prenda-se pouco:** mantenha os papéis como funções simples, e use o framework só para **ligar**. Foi o que fizemos: trocar de framework amanhã custaria só a "cola".
-

Slide 19 — Divisória "Discussão"

15 minutos.

Slide 20 — Para conversar (com respostas esperadas)

1. **"Precisava de um grafo?"** Para a edição semanal, rodando uma vez por semana, o pipeline da Aula 5 quase bastava. O que pesou: execução longa (retomar vale a pena), caminho por confiança e a pausa humana da próxima aula (pausar uma função comum e retomar dias depois é difícil sem checkpoint).
2. **"Que processo cai no meio e recomeça do zero?"** Deixe as pessoas darem exemplos: importação de planilhas grandes, processamento em lote, cadastros com várias etapas.
3. **"Onde um processo precisa lembrar?"** Atendimento (o histórico do cliente), acompanhamento de um pedido, uma conversa com um assistente. A thread seria o cliente, o pedido ou a conversa.
4. **"Quem decide a cota por categoria?"** Uma regra editorial é decisão de negócio (de quem edita a newsletter), não de quem programa. Bom caminho: deixar a regra configurável (`maxPorCategoria`) e documentada, e revisá-la com dados.
5. **"Para que guardar o caminho errado?"** Auditoria (o que aconteceu de verdade), depuração e novos casos de avaliação.

Se o tempo apertar: 1, 2 e 4.

Slide 21 — Para a próxima aula

- **Exercício:** um grafo de triagem de chamados (financeiro, suporte, perguntas frequentes ou uma pessoa), com memória por cliente. O desafio é a viagem no tempo.

- **Projeto final, etapa M6:** o curador como grafo, com cota por categoria e checkpoint. Cada pessoa roda com falha simulada e retoma.

Slide 22 — Próxima aula

Gancho: hoje a fila humana só **anota** os itens incertos. Na Aula 7, o grafo vai **parar** nesses itens e esperar uma pessoa aprovar, editar ou rejeitar, e só então seguir. É o *human-in-the-loop* ("humano no circuito").

Glossário rápido

Termo	Em uma frase
Grafo	caixinhas (nós) ligadas por setas (arestas)
Estado	os dados compartilhados que passam de passo em passo
Nó	um passo do grafo: uma função que recebe o estado e devolve o que mudou
Aresta	a seta: "depois deste, vá para aquele"
Aresta condicional	uma seta que escolhe o próximo passo olhando o estado
Reducer	a regra para juntar um valor novo com o antigo (substituir, acrescentar...)
Fan-out / <code>Send</code>	abrir uma tarefa em várias, uma por item, em paralelo
Nó de junção	um passo que espera todos os ramos paralelos e roda uma vez
Checkpoint	o retrato do estado gravado depois de cada passo
Checkpointer	quem grava os checkpoints (memória, arquivo, banco)
Thread	uma execução com memória própria, identificada por um nome
Retomar	continuar do último checkpoint depois de uma falha
Viagem no tempo	voltar a um checkpoint antigo, mudar um valor e seguir, criando um ramo
Idempotente	rodar duas vezes dá o mesmo resultado que rodar uma
Mermaid	linguagem de texto para desenhar diagramas
Execução durável	um fluxo que sobrevive a quedas porque salva o progresso